Transacciones y control de errores

1.1 Concepto de transacción y propiedades ACID

Una **transacción** es una **unidad lógica de trabajo** que agrupa una o más operaciones sobre la base de datos que deben ejecutarse de forma **atómica**, es decir:

o se realizan todas correctamente, o no se realiza ninguna.

Las transacciones garantizan la **integridad y consistencia** de los datos, especialmente en sistemas donde múltiples usuarios o procesos acceden simultáneamente a la base de datos.

Ejemplo conceptual

Imagina una **transferencia bancaria**:

1. Restar 200 € de la cuenta de origen.
2. Sumar 200 € a la cuenta de destino.

Ambas operaciones deben ejecutarse juntas; si una falla, la otra no debe aplicarse. Por tanto, se agrupan dentro de una **transacción**.

Propiedades ACID

El funcionamiento de las transacciones se basa en cuatro propiedades fundamentales:

Propiedad	Nombre	Descripción
A	*Atomicidad (Atomicity)*	Todas las operaciones dentro de la transacción se ejecutan completamente o ninguna.
C	*Consistencia (Consistency)*	La base de datos pasa de un estado válido a otro, respetando todas las restricciones y reglas de integridad.
I	*Aislamiento (Isolation)*	Las transacciones concurrentes no interfieren entre sí.
D	*Durabilidad (Durability)*	Una vez confirmada (commit), la transacción es permanente, incluso ante fallos del sistema.

Fases de una transacción

Fase	Descripción
Inicio	Se ejecutan las operaciones bajo control transaccional.
Validación	El sistema verifica que no se violen reglas de integridad ni bloqueos.
Confirmación (COMMIT)	Se consolidan los cambios en la base de datos.
Deshacer (ROLLBACK)	Si ocurre un error, se anulan todos los cambios.

Transacciones en MySQL

Por defecto, MySQL (y otros gestores como PostgreSQL) trabajan en modo **autocommit = true**, lo que significa que cada sentencia INSERT, UPDATE o DELETE se confirma automáticamente.

Para usar transacciones, hay que desactivar este modo y confirmar manualmente:

```
SET autocommit = 0;   -- Desactiva el autocommit
START TRANSACTION;    -- Inicia la transacción
UPDATE cuentas SET saldo = saldo - 200 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 200 WHERE id = 2;
COMMIT;               -- Confirma los cambios
-- Si algo falla: ROLLBACK;
```

1.2 Transacciones en Java con JDBC

En JDBC, el control de transacciones se realiza a través del objeto Connection. Cada conexión mantiene su propio contexto transaccional, es decir, todas las operaciones que se ejecutan usando esa conexión pertenecen a la misma transacción mientras no se confirme o se deshaga.

Por defecto, el modo **autocommit** está activado (true), lo que significa que **cada sentencia SQL se confirma automáticamente**.
Si se quiere agrupar varias operaciones para que se ejecuten juntas o se deshagan todas en caso de error, se debe **desactivar el autocommit** y controlar manualmente la transacción.

Pasos básicos para gestionar una transacción

1. **Obtener la conexión** a la base de datos.
2. **Desactivar el autocommit** con setAutoCommit(false).
3. Ejecutar todas las operaciones necesarias (INSERT, UPDATE, DELETE).
4. Si todas tienen éxito, llamar a commit().
5. Si ocurre algún error, hacer rollback().
6. Finalmente, volver a activar el autocommit (setAutoCommit(true)) y cerrar la conexión.

API esencial

- Savepoint sp = con.setSavepoint("NOMBRE");
- con.rollback(sp); ← deshace lo hecho **después** del savepoint
- con.releaseSavepoint(sp); (opcional, libera la marca)
- con.rollback(); ← deshace **toda** la transacción
- con.commit(); ← confirma todo

Nota: revisa en el aula virtual el ejemplo de una transferencia entre dos cuentas.

1.3 Aislamiento de transacciones y anomalías de concurrencia

El **aislamiento** controla cómo se "ven" entre sí las transacciones concurrentes. Su finalidad es evitar resultados incoherentes cuando varias transacciones leen y escriben al mismo tiempo. Forma parte del acrónimo **ACID** que ya mencionamos al inicio. Recordemos:

- **A**tomicidad: todo o nada.
- **C**onsistencia: se mantienen las invariantes del dominio (por ejemplo, "el saldo no puede ser negativo").
- Aislamiento: minimiza los efectos de la concurrencia (qué ve A mientras B modifica).
- **D**urabilidad: lo confirmado persiste aunque haya fallos

```
con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
con.setAutoCommit(false);
// ... trabajo ...
```

Fenómenos (anomalías) típicos

Los niveles de aislamiento controlan si estos fenómenos pueden pasar:

- **Lectura sucia (Dirty Read)**: una transacción lee datos **no confirmados** de otra.
- **Lectura no repetible (Non-repeatable Read)**: una fila leída cambia entre dos lecturas dentro de la misma transacción (otra transacción la actualizó y confirmó).
- **Lecturas fantasma (Phantom Read)**: reaparecen/desaparecen filas que cumplen una **condición** de búsqueda (otra transacción insertó/eliminó filas del rango).
- **Dirty Write (escritura sucia)**: dos transacciones escriben sobre los cambios **no confirmados** de la otra (los motores serios la evitan siempre).

Niveles de aislamiento

JDBC / ANSI SQL	¿Lectura sucia?	¿No repetible?	¿Fantasmas?	Uso típico
READ_UNCOMMITTED	✓	✓	✓	Pruebas, casi nunca en producción
READ_COMMITTED	✗	✓	✓	Equilibrio práctico (defecto típico en PostgreSQL)
REPEATABLE_READ	✗	✗	✓	Mayor estabilidad de lectura (defecto típico en MySQL/InnoDB)
SERIALIZABLE	✗	✗	✗	Máxima integridad; menor concurrencia

Constantes JDBC (por claridad):
`Connection.TRANSACTION_READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE`

Elegir el nivel adecuado

- **READ_COMMITTED**: por defecto razonable para la mayoría de CRUD. Evita lecturas sucias con buena concurrencia.
- **REPEATABLE_READ**: informes que leen varias veces las **mismas filas** y requieren consistencia intra-transacción (p. ej., recalcular totales).
- **SERIALIZABLE**: reglas complejas con dependencias entre lecturas y escrituras de **rangos** (riesgo de "fantasmas" perjudiciales) o cuando necesitas garantías máximas y aceptas menos paralelismo.
- **Transacciones cortas y acotadas**: sea cual sea el nivel, mantenlas **breves** y con **buenos índices** en las columnas de filtro para minimizar bloqueos.

En el ejemplo **Tx_TransferenciaSimple** hemos usado:

- READ_COMMITTED para evitar lecturas sucias y mantener buen rendimiento.
- Actualizaciones **atómicas** con saldo = saldo - ? y condición AND saldo >= ? para evitar descubiertos **sin** pasos previos vulnerables
- Frontera transaccional clara (un solo commit() o rollback()).

1.4 Savepoint

Un *savepoint* es una "marca" dentro de una transacción para poder **deshacer solo una parte** con rollback(savepoint) sin perder todo lo anterior.
Útil cuando tu caso de uso tiene **pasos independientes** donde algunos pueden conservarse aunque falle un paso posterior.

En el ejemplo de **transferencia bancaria** no tiene sentido: debe ser **todo-o-nada**.

Sin embargo, el ejemplo de validar un **pedido** y luego crear su **albarán**. Si falla el albarán, **conservo** el pedido validado (rollback parcial).

Nota: Revisa el ejemplo del aula virtual

Anexo: algunos tips o "mini-plantillas"

Transacción segura

Cuándo usarlo: siempre que agrupes **varias sentencias DML** que deben ser "todo-o-nada". Este patrón garantiza una **frontera transaccional clara** (setAutoCommit(false) → commit/rollback) y deja la conexión **limpia** al final (restaura el autoCommit), algo crucial si usas pools de conexiones. Úsalo como plantilla base en cualquier caso de uso CRUD serio.

```
try (Connection con = ConexionBD.conectar()) {
    con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
    boolean prev = con.getAutoCommit();
    con.setAutoCommit(false);
    try {
        // ... SQL crítico con PreparedStatement ...
        con.commit();
    } catch (SQLException e) {
        try { con.rollback(); } catch (SQLException rb) {
e.addSuppressed(rb); }
        throw e;
    } finally {
        try { con.setAutoCommit(prev); } catch (SQLException ignore) {}
    }
}
```

Actualización atómica

Por qué así y no SELECT + UPDATE: expresar la regla de negocio **dentro del UPDATE** (con saldo = saldo - ? y AND saldo >= ?) hace la operación **atómica en la BD** y evita condiciones de carrera entre "leer-comprobar-actualizar". Si no hay fondos, el UPDATE afecta **0 filas** y lo puedes detectar desde Java para **hacer rollback()** y mostrar un mensaje claro. Este patrón es el que usa en **Tx_TransferenciaSimple**.

```
UPDATE cuentas
SET saldo = saldo - ?
WHERE id = ? AND saldo >= ?;   -- evita descubierto y condiciones de carrera
```

Asignación parámetros en Java:

```
ps.setBigDecimal(1, importe); // - ?
ps.setInt(2, idOrigen);       // id = ?
ps.setBigDecimal(3, importe); // saldo >= ?
```

Control de intentos

Cuándo aplicar reintentos: algunos fallos son **transitorios** (p. ej., *deadlocks* o conflictos de serialización). Un **reintento limitado** con pequeña **espera incremental** suele resolverlos sin afectar al usuario. Úsalo **solo** para errores de concurrencia (p. ej., SQLTransactionRollbackException) y **no** para errores lógicos (FK, formato, etc.). Mantén MAX bajo (2–3) para no sobrecargar el sistema.

```
int intentos = 0, MAX = 3;
while (true) {
  try (Connection con = ConexionBD.conectar()) {
    con.setAutoCommit(false);
    // ... trabajo ...
    con.commit();
    break;
  } catch (SQLTransactionRollbackException e) { // deadlock/serialización
    if (++intentos >= MAX) throw e;
    try { Thread.sleep(200L * intentos); } catch (InterruptedException
ignored) {}
  }
}
```

Savepoint

Idea práctica: marca un **punto útil** dentro de la transacción para poder **revertir solo lo posterior** (rollback(savepoint)). Es ideal en flujos **multi-paso** donde un primer paso deja el sistema en un estado válido (p. ej., *pedido VALIDADO*) y un paso posterior es deseable pero no imprescindible (crear **albarán**). **No** lo uses en operaciones **indivisibles** por negocio (transferencias, asientos contables): ahí todo va en una sola commit/rollback

```
Savepoint sp = con.setSavepoint("NOMBRE");
// ...
con.rollback(sp);  // deshacer parcial
// ...
con.commit();      // confirma lo conservado
```